

University of Salford, MSc Data Science

Module: Machine Learning & Data Mining

Date: Trimester 1, 2024-2025

Session: Workshop Week 8

Topic: Neural Networks in Azure Machine Learning Designer

Tools: Azure Machine Learning Designer

Objectives:

After completing this workshop, you will be able to:

- Create an Azure for Students account and set up a machine learning workspace and associated cluster compute resources
- Use Machine Learning Studio to create data assets
- Use Machine Learning Designer to create and run machine learning pipelines using the data assets you have created
- Create a Neural Network classification model within Machine Learning Designer
- Assess and evaluate model performance
- Create a real-time inference pipeline, and deploy it as a service for client applications



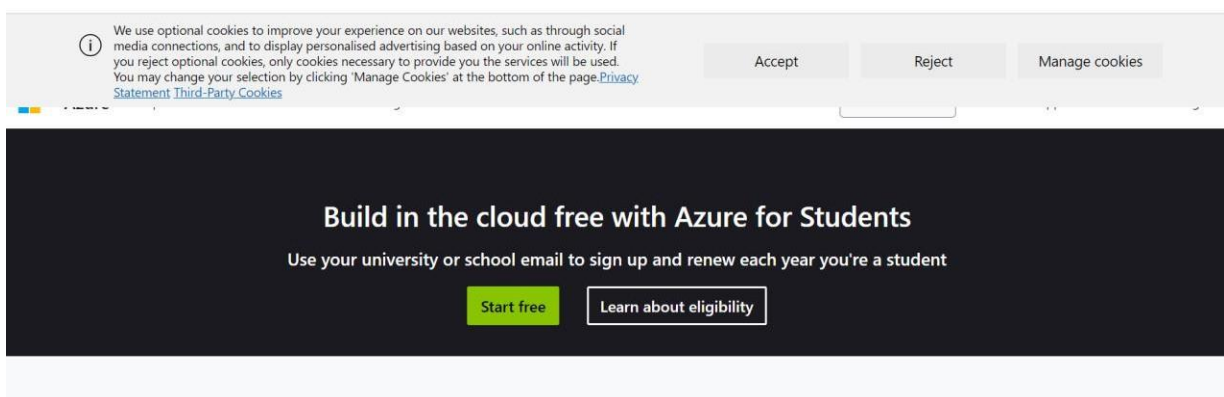
Creating Azure for Students Account

Azure for Students offers a license with \$100 in Azure credits that can be used for the first 12 months along with some free services without the need to provide a credit card when subscribing.

Below, it is shown step by step how to activate an Azure for Students subscription.

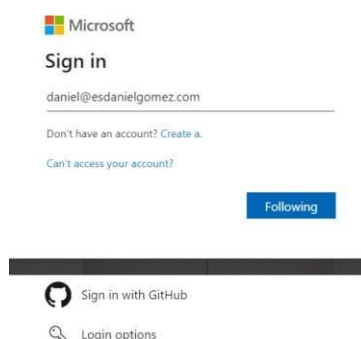
Step 1: Access the Azure for Students page.

Link: <https://azure.microsoft.com/en-gb/free/students/>



Step 2: Identify yourself.

You then need to sign in or sign up with a Microsoft account. For this step you can use an institutional email or a personal email.



Note: In case an institutional account has several purposes, you must select: Personal account and always use this means to access to Azure.

Step 3: Academic verification.

As a first point, we will need to verify our authenticity with a verification code that will be sent by text message. Then we must perform the academic verification, in this case with an email (.edu) as you can see below:

Identity verification by phone

Student Verification

Academic verification required

The account you are logged in with has not yet been verified to access the benefits of the offer. Use the form below to perform the verification.

Verification method

Educational email address

Enter your educational email address. If the academic institution is in our database, we will send you an email with a verification link.

The educational email address will only be used for verification processes. For all other features, use your Microsoft account email.

Educational email address

daniel.gomez@██████████.edu.ec

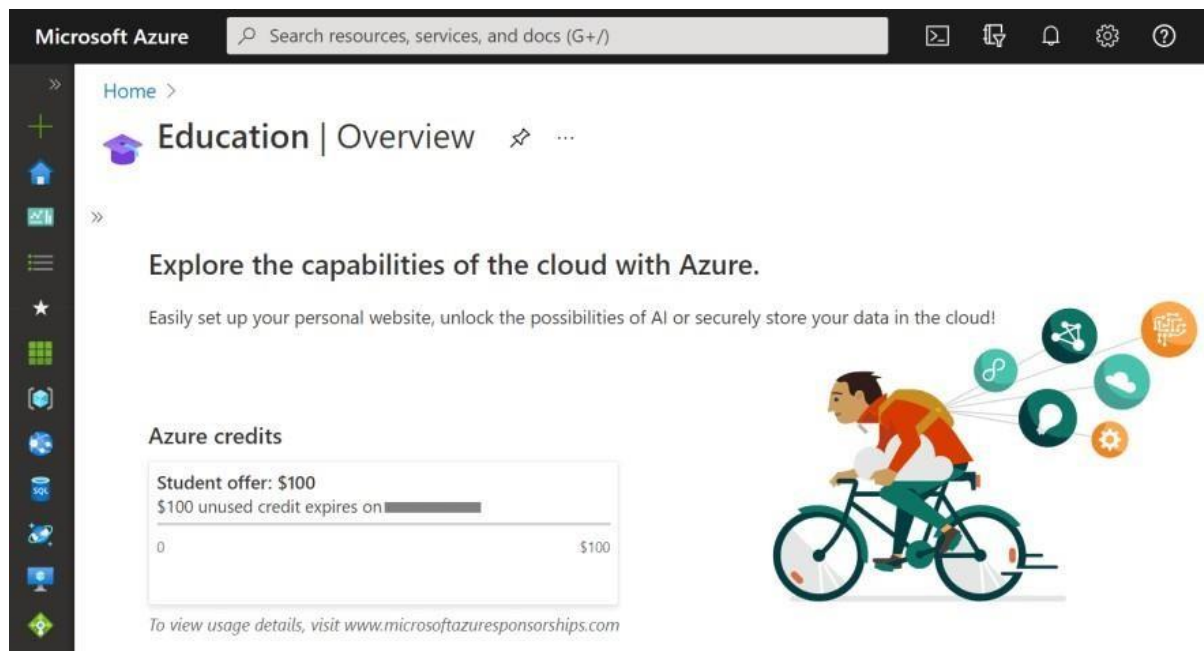
Retype your educational email address.

daniel.gomez@██████████.edu.ec

Check academic status

Step 2: Identify yourself.

After this process you will be redirected to the azure portal home page:



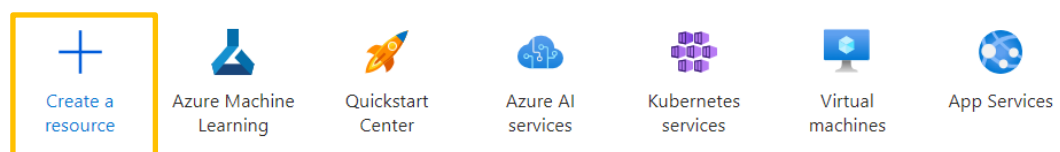
With this you are ready to start using Azure.

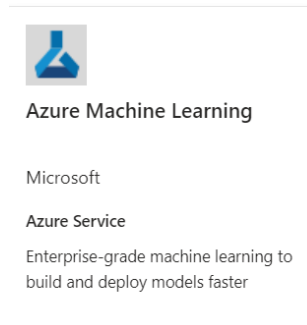
Note To complete this lab, you will need an [Azure subscription](#) in which you have administrative access.

Create an Azure Machine Learning workspace

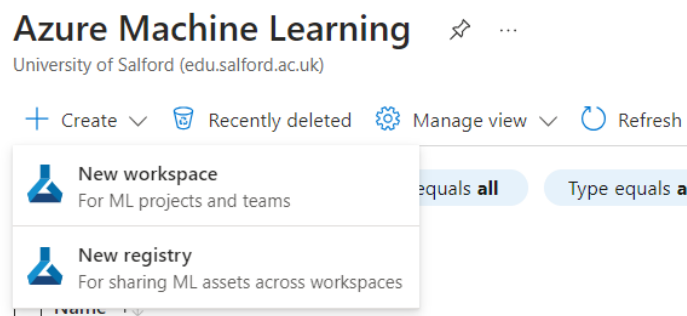
1. Click on this link [Azure portal](#) and sign into Azure using your Microsoft credentials.
2. Select “+ **Create a resource**”, search for **Machine Learning**, and create a new **Azure Machine Learning** resource with an *Azure Machine Learning* plan.

Azure services





And then create a new workspace:



3. Use the following settings:

- **Subscription:** *Your Azure subscription (Azure for Students)*
- **Resource group:** *Create or select a resource group.*
- **Workspace name:** *Enter a unique name for your workspace.*
- **Region:** *Select the closest geographical region (UK west)*
- **Storage account:** *Note the default new storage account that will be created for your workspace.*
- **Key vault:** *Note the default new key vault that will be created for your workspace.*
- **Application insights:** *Note the default new application insights resource that will be created for your workspace.*
- **Container registry:** *None (one will be created automatically the first time you deploy a model to a container)*

4. Click on **Review + create** button, then select **Create**. Wait for your workspace to be created (it can take a few minutes),

5. Click on “Go to resource”

Home >

Microsoft.MachineLearningServices | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

- Overview
- Inputs
- Outputs
- Template

✓ Your deployment is complete

Deployment name: Microsoft.MachineLearn... Start time: 11/2/2023, 7:56:58 PM
Subscription: [Azure for Students](#) Correlation ID: 2834b664-4eb3-4707-b918-0f56816ab
Resource group: [w8](#)

Deployment details

Next steps

[Go to resource](#)

Give feedback

[Tell us about your experience with deployment](#)

6. Scroll down in the page and select **Launch studio** (or open a new browser tab and navigate to <https://ml.azure.com>, and sign into Azure Machine Learning studio using your Microsoft account).

week8 ☆ ...

Azure Machine Learning workspace

Search

Download config.json Delete

workspace: [week81006958084](#)

Application insights: [week82312830719](#)

MLflow tracking URI: [azureml://ukwest.api.azureml.ms/mlflow/v1.0/subscriptions/63fae860-a...](#)

Alerts

Metrics

Diagnostic settings

Logs

Automation

CLI / PS

Tasks (preview)

Export template

Support + troubleshooting

Resource health

Usage + quotas

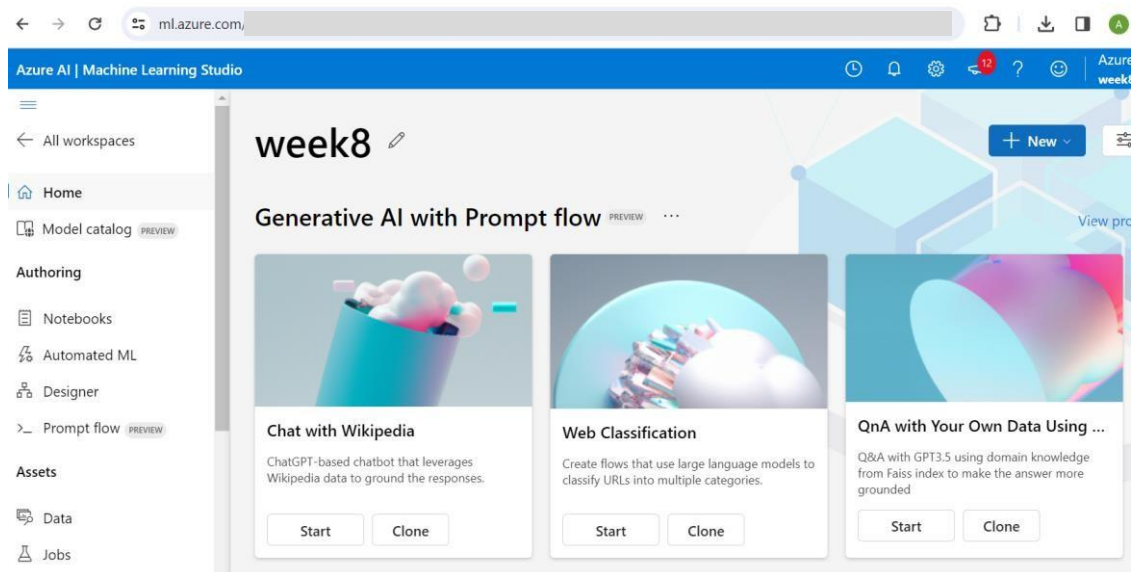
Support + Troubleshooting

Work with your models in Azure Machine Learning Studio

The Azure Machine Learning Studio is a web app where you can build, train, test, and deploy ML models. Launch it now to start exploring, or [learn more about the Azure Machine Learning studio](#)

[Launch studio](#)

7. In Azure Machine Learning studio, you should see your newly created workspace. If that is not the case, from the left-hand menu select **Workspaces**, where all the workspaces associated to your subscription are listed. Choose the one you created for this exercise.



Note If you are using your own Azure subscription, you may consider creating the workspace once and reusing it in other modules. Your Azure subscription will be charged a small amount for data storage as long as the Azure Machine Learning workspace exists in your subscription, so **we recommend you delete the Azure Machine Learning workspace when it is no longer required.**

Create compute

1. In [Azure Machine Learning studio](#), select the three lines at the top left to view the various pages in the interface (you may need to maximize the size of your screen). You can use the pages in the left hand pane to manage the resources in your workspace. Select the **Compute** page (under **Manage**).
2. On the **Compute** page, select the **Compute clusters** tab, and add a new compute cluster (Click on +New) with the following settings. You'll use this to train a machine learning model:
 - **Location:** *Select the same as your workspace. If that location is not listed, choose the one closest to you.*
 - **Virtual machine tier:** Dedicated
 - **Virtual machine type:** CPU
 - **Virtual machine size:**
 - Choose **Select from all options**
 - Search for and select **Standard_DS11_v2**
 - Select **Next**
 - **Compute name:** *enter a unique name.*
 - **Minimum number of nodes:** 0

- **Maximum number of nodes:** 2
- **Idle seconds before scale down:** 120
- **Enable SSH access:** Clear (off)
- Select **Create**

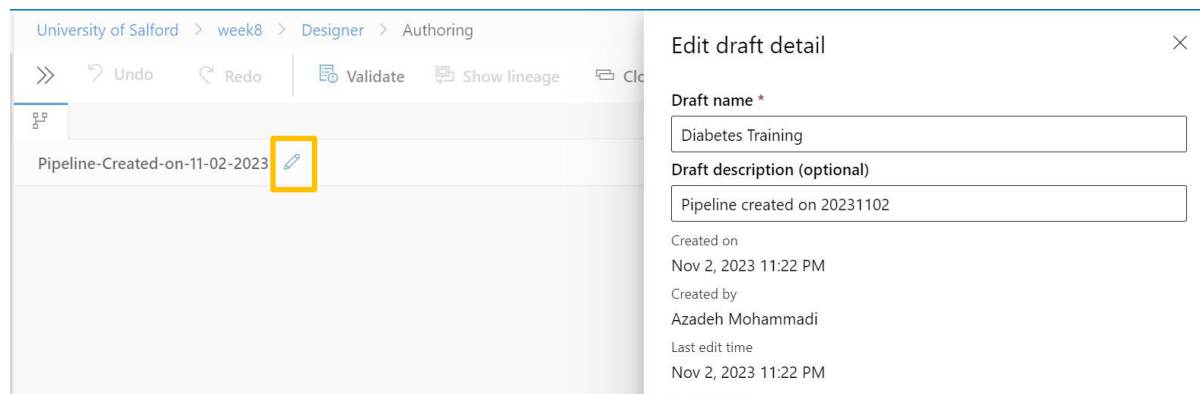
Note Compute instances and clusters are based on standard Azure virtual machine images. For this module, the *Standard_DS11_v2* image is recommended to achieve the optimal balance of cost and performance. If your subscription has a quota that does not include this image, choose an alternative image; but bear in mind that a larger image may incur higher cost and a smaller image may not be sufficient to complete the tasks.

The compute cluster will take some time to be created. You can move onto the next step while you wait.

Create a pipeline in Designer

To get started with Azure Machine Learning designer, first you must create a pipeline and add the dataset you want to work with.

1. In [Azure Machine Learning studio](#), expand the left pane by selecting the three lines at the top left of the screen. View the **Designer** page (under **Authoring**), and select + to create a new pipeline.
2. Click on pen icon near the name of the pipeline.
3. Under **Draft name**, change the draft name (**Pipeline-Created-on-date**) to **Diabetes Training**.
4. Click on **Save**.

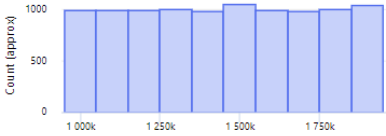
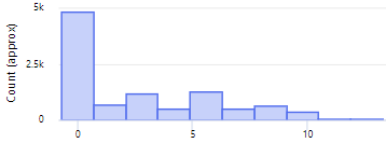
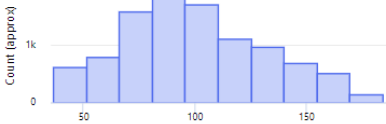


Create a dataset

1. In [Azure Machine Learning studio](#), expand the left pane by selecting the three lines at the top left of the screen. View the **Data** page (under **Assets**). The Data page contains specific data files or tables that you plan to work with in Azure ML. You can create datasets from this page as well.
2. Click on “+Create” and create a dataset using the following settings:
 - **Basic Info:**
 - **Name:** diabetes-data
 - **Description:** Diabetes data
 - **Dataset type:** Tabular
 - 3. Select **Next** and then **from web files:**
 - **Web URL:**
media.githubusercontent.com/media/microsoft/MLOps/refs/heads/master/examples/AzureML-Primers/data/diabetes.csv
 - **Skip data validation:** *Do not select*
Click on Next
 - **Settings and preview:**
 - **File format:** Delimited
 - **Delimiter:** Comma
 - **Encoding:** UTF-8
 - **Column headers:** Only first file has headers
 - **Skip rows:** None
 - **Dataset contains multi-line data:** *do not select*
 - Click on Next
 - **Schema:**
 - Include all columns other than **Path**
 - Review the automatically detected types
 - Click on Next
 - Click on Create
4. After the dataset has been created, open it and view the **Explore** tab to see a sample of the data. This data represents details from patients who have been tested for diabetes.

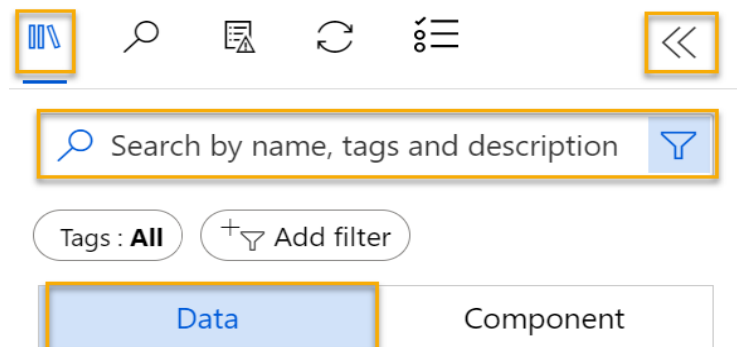
Preview Profile							
Number of columns: 10 Number of rows: First 50							
PatientID	Pregnancies	PlasmaGlucose	DiastolicBloodPress...	TricepsThickness	SerumInsulin	BMI	DiabetesPedigree
1698452	3	152	47	44	79	18.766	0.157
1647913	0	85	81	31	21	21.364	0.303
1103142	2	104	77	47	70	23.393	0.174
1798712	7	108	95	10	232	19.235	0.125
1174304	2	118	61	56	66	35.991	0.125

Click on **Profile** to view the distribution of various columns and detailed statistics about the data. (Exploratory Data Analysis)

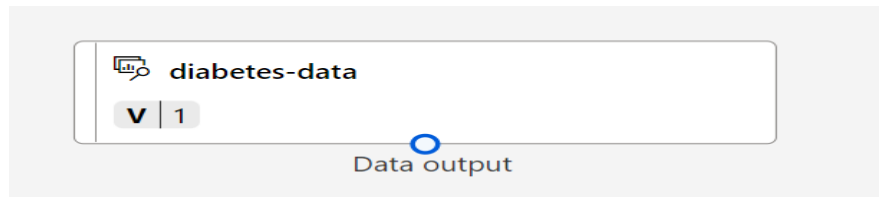
Number of columns: 10 Number of rows: 10000						
Column ▾	Profile	Type ▾	Min ▾	Max ▾	Count ▾	Missing count ▾
PatientID		Integer	1000038	1999997	10000	0
Pregnancies		Integer	0	14	10000	0
PlasmaGlucose		Integer	44	192	10000	0

Load data to canvas

1. Return to your pipeline by selecting **Designer** on the left-hand menu **under Authoring**. On the **Designer** page, select the **Diabetes Training** pipeline by clicking on it.
2. Then select the arrows icon (>>) at top left to expand the panel if it is not already expanded. The panel should open by default to the **Asset library** pane, indicated by the books icon at the top of the panel. Note that there is a search bar to locate assets. Notice two buttons, **Data** and **Component**.



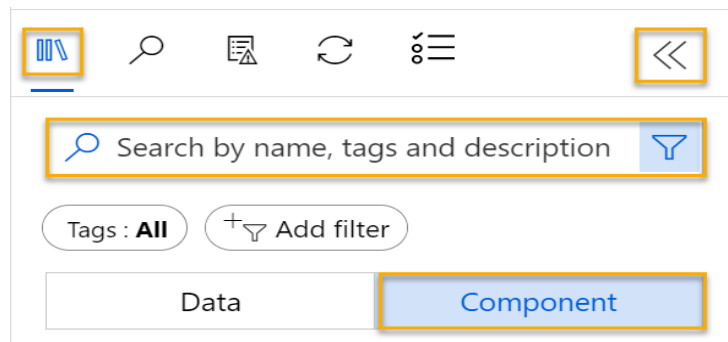
3. Click on **Data**. Search for the **diabetes-data** dataset and place it onto the canvas by drag and drop.
4. Right-click (Ctrl+click on a Mac) the **diabetes-data** dataset on the canvas, and click on **Preview data**.
5. Review the schema of the data in the *Profile* tab, noting that you can see the distributions of the various columns as histograms.
6. Scroll down and select the column heading for the **Diabetic** column, and note that it contains two values **0** and **1**. These values represent the two possible classes for the *label* that your model will predict, with a value of **0** meaning that the patient does not have diabetes, and a value of **1** meaning that the patient is diabetic.
7. Scroll back up and review the other columns, which represent the *features* that will be used to predict the label. Note that most of these columns are numeric, but each feature is on its own scale. For example, **Age** values range from 21 to 77, while **DiabetesPedigree** values range from 0.078 to 2.3016. When training a machine learning model, larger values can sometimes dominate the resulting predictive function, thereby reducing the influence of features on a smaller scale. Typically, data scientists mitigate this possible bias by normalizing the numeric columns so they're on the similar scales.
8. Close the **diabetes-data result visualization** tab so that you can see the dataset on the canvas like this:



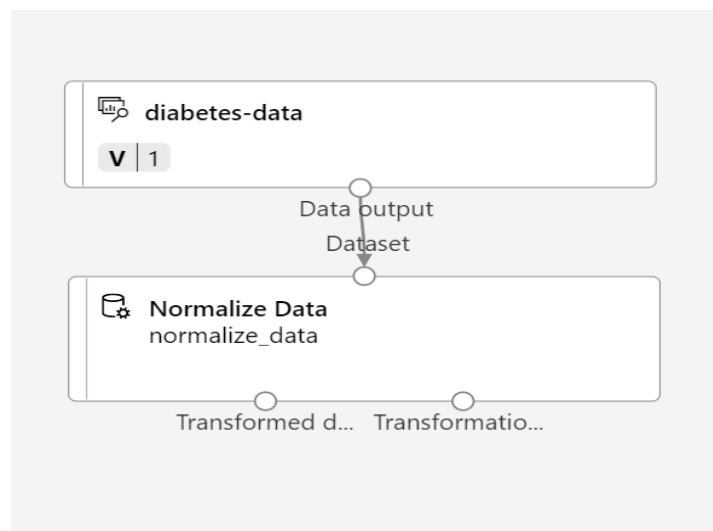
Add transformations

Before you can train a model, you typically need to apply some pre-processing transformations to the data.

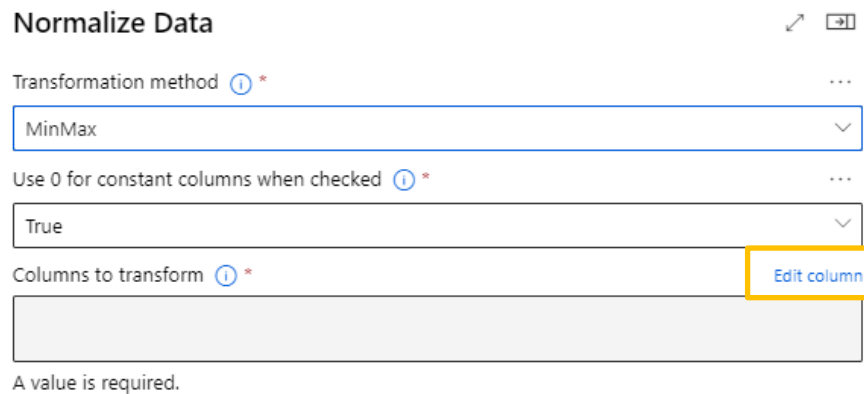
1. In the **Asset library** pane on the left, click on **Component**, which contain a wide range of modules you can use for data transformation and model training. You can also use the search bar to quickly locate modules.



2. Find the **Normalize Data** module and place it on the canvas (drag and drop), below the **diabetes-data** dataset. Then connect the output from the bottom of the **diabetes-data** dataset to the input at the top of the **Normalize Data** module, like this:



3. Double-click the **Normalize Data** module to view its settings, noting that it requires you to specify the transformation method and the columns to be transformed.
4. Set the *Transformation method* to **MinMax** and the *Use 0 for constant columns when checked* to **True**.
5. Click on “Edit column” and then you can add the following columns, it is enough to select column name from the list



Normalize Data

Transformation method ⓘ *

MinMax

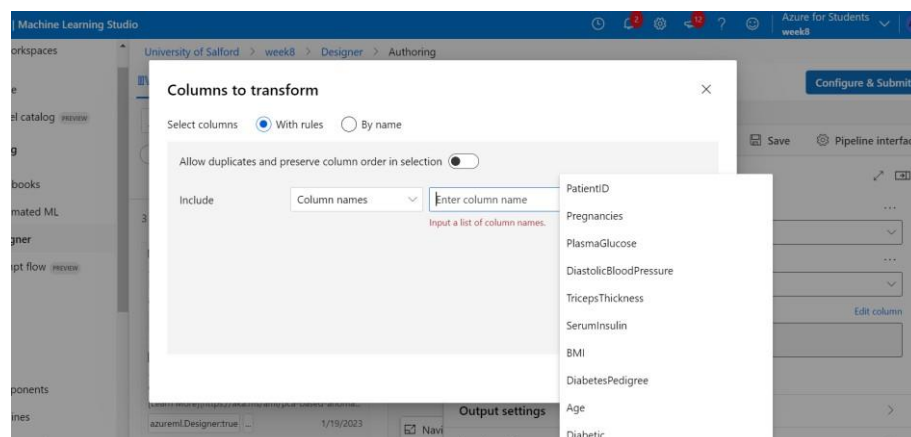
Use 0 for constant columns when checked ⓘ *

True

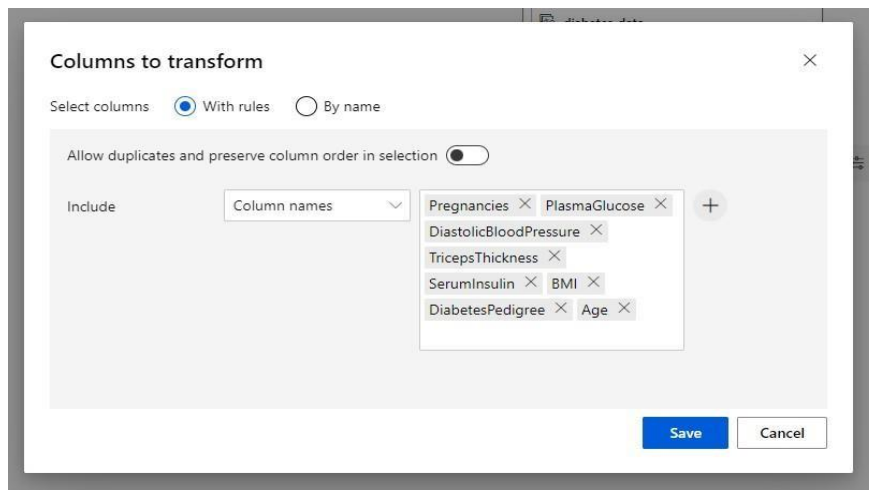
Columns to transform ⓘ *

[Edit column](#)

A value is required.



6. Edit the columns to include the following column names, as shown in the image:
 - **Pregnancies**
 - **PlasmaGlucose**
 - **DiastolicBloodPressure**
 - **TricepsThickness**
 - **SerumInsulin**
 - **BMI**
 - **DiabetesPedigree**
 - **Age**



And click on Save.

The data transformation is normalizing the numeric columns to put them on the same scale, which should help prevent columns with large values from dominating model training. You'd usually apply a whole bunch of pre-processing transformations like this to prepare your data for training, but we'll keep things simple in this exercise.

Run the pipeline

To apply your data transformations, you need to run the pipeline as an experiment.

1. Click on “Configure and Submit” on top right, and **create a new experiment** and call it “**mslearn-diabetes-training**” and click on Next.



For Runtime setting, select computer cluster you had created before and click on Review+Submit and then click Submit.

Set up pipeline job

Basics

Experiment name

☐ Select existing ☒ Create new

New experiment name *

mslearn-diabetes-training

Job display name

Diabetes Training

Job description

Pipeline created on 20231102

Job tags

Review + Submit Back Next Close

2. **Wait a few minutes for the run to finish.** You can choose pipeline from left hand menu and see the status of pipeline. You should wait until status changes from Running to Completed. It will take few minutes.

View the transformed data

1. When the run has completed, click on Jobs from left hand panel (under Assests) and find mslearn-diabetes-training and click on the latest job.

Jobs		
All experiments All jobs All schedules		
Refresh Archive experiment View options		
Search Columns		
Experiment	☆ Latest job	Last submitted ↓
mslearn-diabetes-training	Diabetes Training	Nov 3, 2023 7:14 PM
dataset_profile	lemon_honey_ltv070xp	Nov 2, 2023 10:48 PM

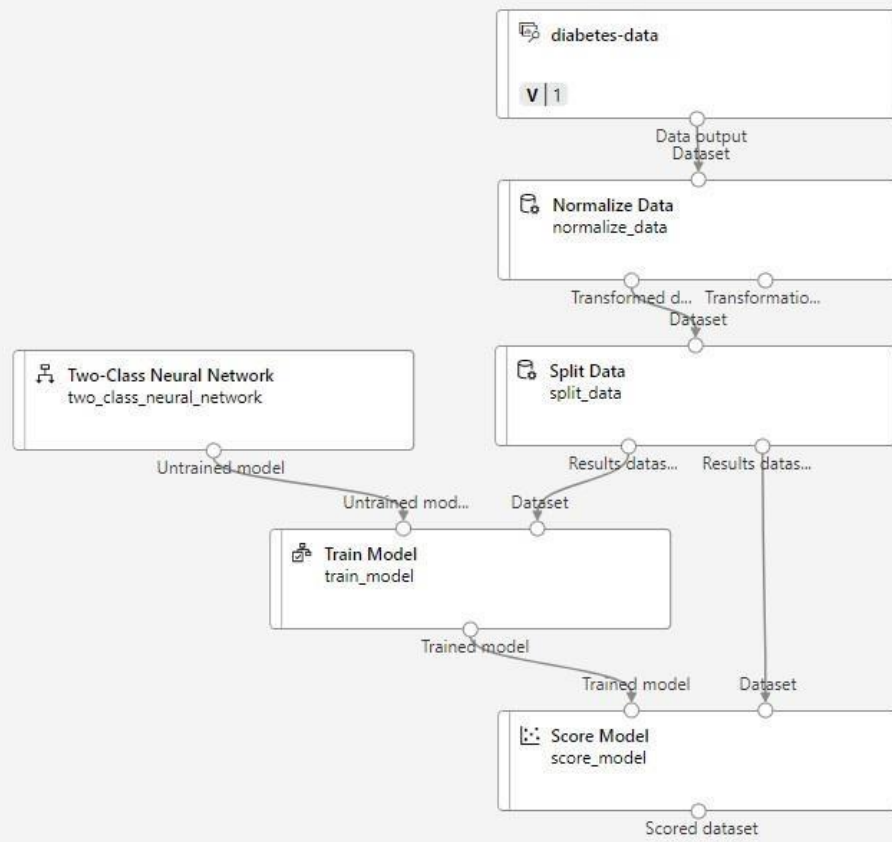
2. Right-click (Ctrl+click on a Mac) the **Normalize Data** module on the canvas, and click on **Preview data**. Select **Transformed dataset**.
3. View the data, noting that the numeric columns you selected have been normalized to a common scale.
4. Close the normalized data result visualization. Return to the previous tab by clicking on 

After you've used data transformations to prepare the data, you can use it to train a machine learning model.

Add training modules

It's common practice to train the model using a subset of the data, while holding back some data with which to test the trained model. This enables you to compare the labels that the model predicts with the actual known labels in the original dataset.

In this exercise, you're going to work through steps to extend the **Diabetes Training** pipeline as shown here:



Follow the steps below, using the image above for reference as you add and configure the required modules.

1. From Designer pane on the left, select **Diabetes Training** and open it.
2. In the Asset library pane, in **Component tab**, search for and place a **Split Data** module onto the canvas under the **Normalize Data** module. Then connect the *Transformed Dataset* (left) output of the **Normalize Data** module to the input of the **Split Data** module.

Tip Use the search bar to quickly locate modules.

3. Double click on **Split Data** module, and configure its settings as follows:
 - **Splitting mode:** Split Rows
 - **Fraction of rows in the first output dataset:** 0.7
 - **Randomized split:** True
 - **Random seed:** 123
 - **Stratified split:** False
4. In the **Asset library**, search for and place a **Train Model** module to the canvas, under the **Split Data** module. Then connect the *Results dataset1* (left) output of the **Split Data** module to the *Dataset* (right) input of the **Train Model** module.
5. The model we're training will predict the **Diabetic** value, so double click on the **Train Model** module and modify its settings to set the **Label column** to **Diabetic**.

The **Diabetic** label the model will predict is a class (0 or 1), so we need to train the model using a *classification* algorithm. Specifically, there are two possible classes, so we need a *binary classification* algorithm.

6. In the **Asset library**, search for and place a **Two-Class Neural Network** module to the canvas, to the left of the **Split Data** module and above the **Train Model** module. Then connect its output to the **Untrained model** (left) input of the **Train Model** module.

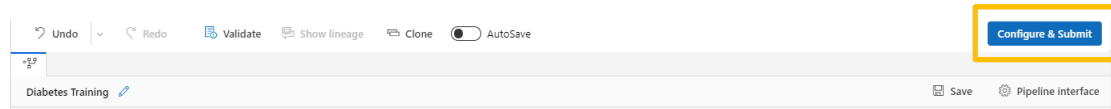
To test the trained model, we need to use the model to *score* the test dataset we held back when we split the original data - in other words, predict labels for the features in the test dataset.

7. In the **Asset library**, search for and place a **Score Model** module to the canvas, below the **Train Model** module. Then connect the output of the **Train Model** module to the **Trained model** (left) input of the **Score Model** module; and connect the **Results dataset2** (right) output of the **Split Data** module to the **Dataset** (right) input of the **Score Model** module.

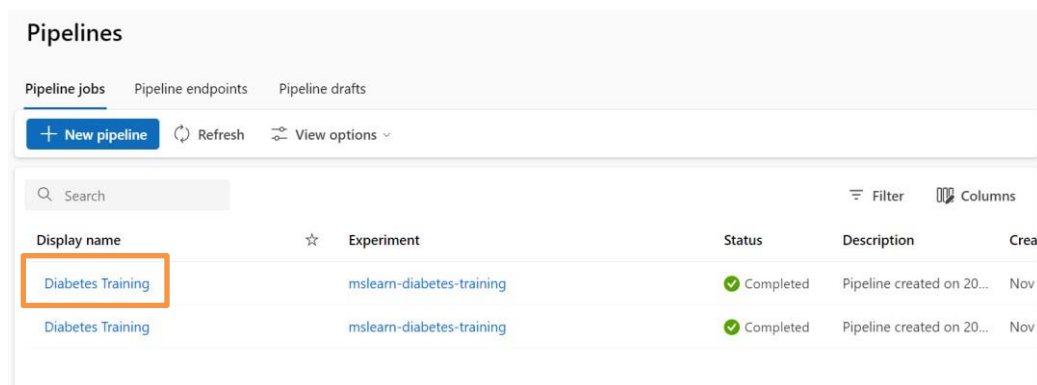
Run the training pipeline

Now you're ready to run the training pipeline and train the model.

1. Click on ... at top right and select **Configure & Submit**, and run the pipeline using the existing experiment named **mslearn-diabetes-training**. (Select existing> mslearn-diabetes-training)



2. Click on Review & Submit and then submit. Wait for the experiment run to finish. This may take 5 minutes or more.
3. When the experiment run has finished, select **pipeline** from left hand panel and click on the last display name.



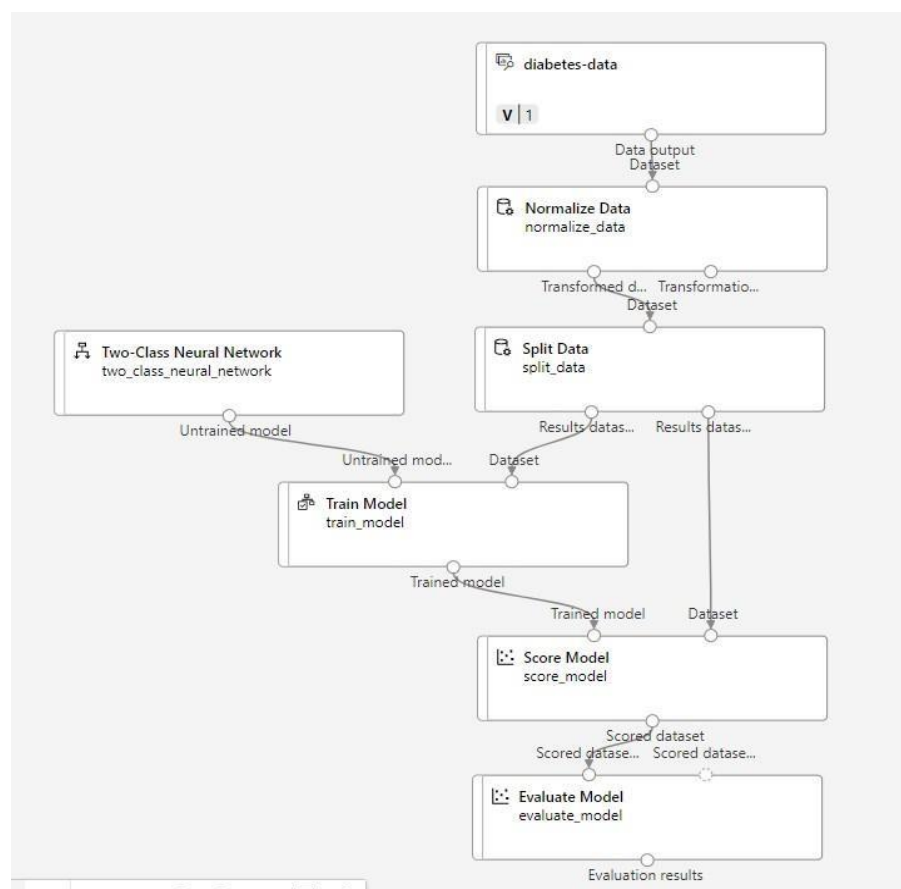
4. On the new tab, right-click (Ctrl+click on a Mac) the **Score Model** module on the canvas, and click on **Preview data**. Select **Scored dataset** to view the results.
5. Scroll to the right, and note that next to the **Diabetic** column (which contains the known true values of the label) there is a new column named **Scored Labels**, which contains the predicted label values, and a **Scored Probabilities** column containing a probability value between 0 and 1. This indicates the probability of a *positive* prediction, so probabilities greater than 0.5 result in a predicted label of **1** (diabetic), while probabilities between 0 and 0.5 result in a predicted label of **0** (not diabetic).
6. Close the **Score Model result visualization** tab by clicking on .

The model is predicting values for the **Diabetic** label, but how reliable are its predictions? To assess that, you need to evaluate the model.

The test data you held back and used to score the model includes the known values for the label. So, to validate the model, you can compare the true values for the label to the label values that were predicted when you scored the test dataset. Based on this comparison, you can calculate various metrics that describe how well the model performs.

Add an Evaluate Model module

1. Open the **Diabetes Training** pipeline you created from **Designer** menu on the left hand panel.
2. In the **Asset library**, search for and place an **Evaluate Model** module to the canvas under the **Score Model** module, and connect the output of the **Score Model** module to the **Scored dataset** (left) input of the **Evaluate Model** module.
3. Ensure your pipeline looks like this:



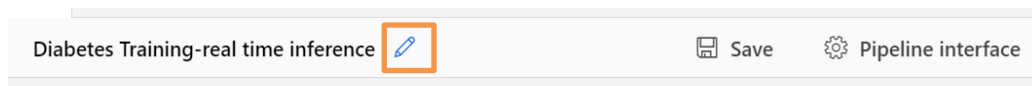
4. Click on ... from top right and select **Configure & Submit**, and then submit to run the pipeline using the existing experiment named **mslearn-diabetes-training**.
5. Wait for the experiment run to finish.
6. When the experiment run has finished, select pipeline from left hand panel and click on the last display name.

7. On the new tab, right-click (Ctrl+click on a Mac) the **Evaluate Model** module on the canvas, and click on **Preview data**. Select **Evaluation results** to view the performance metrics. These metrics can help data scientists assess how well the model predicts based on the test data.
8. Scroll down to view the *confusion matrix* for the model. Observe the predicted and actual value counts for each possible class.
9. Review the metrics to the left of the confusion matrix, which include:
 - **Accuracy:** In other words, what proportion of diabetes predictions did the model get right?
 - **Precision:** In other words, out of all the patients that *the model predicted* as having diabetes, the percentage of time the model is correct.
 - **Recall:** In other words, out of all the patients *who actually have* diabetes, how many diabetic cases did the model identify correctly?
 - **F1 Score**
10. Use the **Threshold** slider located above the list of metrics. Try moving the threshold slider and observe the effect on the confusion matrix. If you move it all the way to the left (0), the Recall metric becomes 1, and if you move it all the way to the right (1), the Recall metric becomes 0.
11. Look above the Threshold slider at the **ROC curve** and **AUC** metric listed with the other metrics below. To get an idea of how this area represents the performance of the model, imagine a straight diagonal line from the bottom left to the top right of the ROC chart. This represents the expected performance if you just guessed or flipped a coin for each patient - you could expect to get around half of them right, and half of them wrong, so the area under the diagonal line represents an AUC of 0.5. If the AUC for your model is higher than this for a binary classification model, then the model performs better than a random guess.
12. Close the **Evaluate Model result visualization** tab.

NB - *The performance of this model isn't all that great, partly because we performed only minimal feature engineering and pre-processing. If you wish to experiment further, you could try a different classification algorithm, such as **Two-Class Decision Forest**, and compare the results. You can connect the outputs of the **Split Data** module to multiple **Train Model** and **Score Model** modules, and you can connect a second **Score Model** module to the **Evaluate Model** module to see a side-by-side comparison. The point of the exercise is simply to introduce you to classification and the Azure Machine Learning designer interface, not to train a perfect model!*

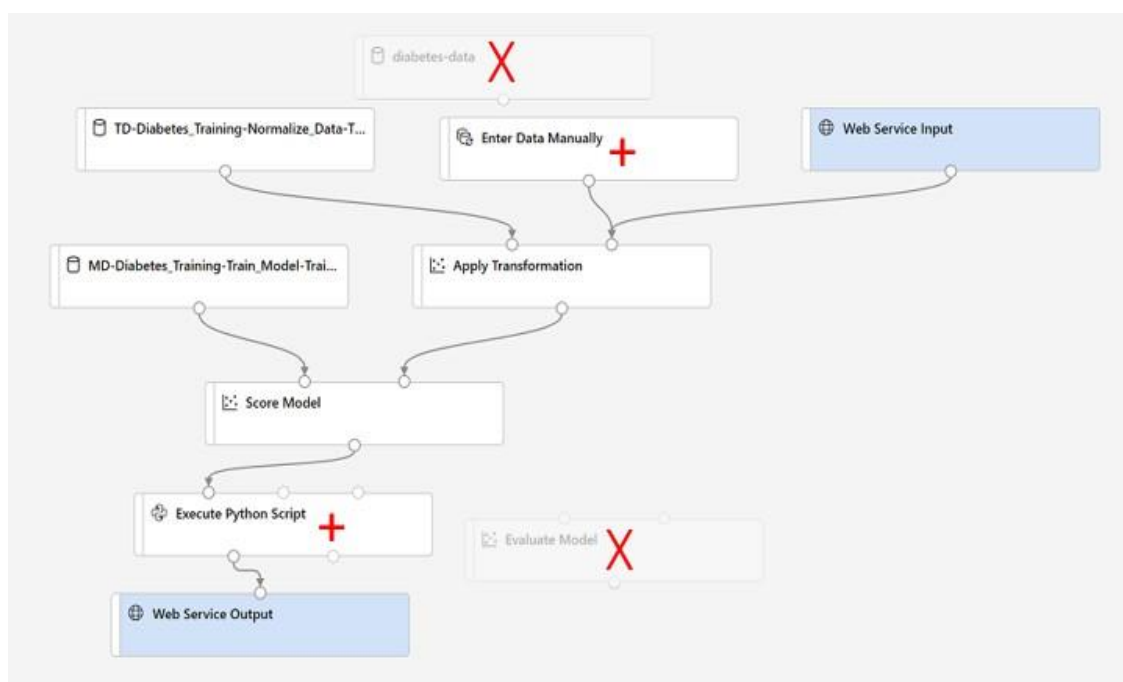
Create an inference pipeline

1. In Azure Machine Learning studio, expand the left-hand pane by selecting the three lines at the top left of the screen. Click on **Jobs** (under **Assets**) to view all of the jobs you have run. Select the experiment **mslearn-diabetes-training**, then select the **Diabetes Training** pipeline.
2. Select **Create inference pipeline**. You may need to expand your screen to full and click on the three dots icon ... on the top right hand corner of the screen in order to find **Create inference pipeline** in the menu.
3. In the **Create inference pipeline** drop-down list, click **Real-time inference pipeline**. After a few seconds, a new version of your pipeline named **Diabetes Training-real time inference** will be opened.
4. Click on pen icon to change settings.



5. Under **Draft details**, rename the new pipeline to **Predict Diabetes**, and then save it. This new pipeline contains a web service input for new data to be submitted, and a web service output to return results. Some of the transformations and training steps are a part of this pipeline. The trained model will be used to score the new data.

You're going to make the following changes to the inference pipeline:



You should add/remove some components:

- Replace the **diabetes-data** dataset with an **Enter Data Manually** module that doesn't include the label column (**Diabetic**). You may also need to add **Web Service Input**
 - Remove the **Evaluate Model** module.
 - Insert an **Execute Python Script** module before the web service output to return only the patient ID, predicted label value, and probability.
6. The inference pipeline assumes that new data will match the schema of the original training data, so the **diabetes-data** dataset from the training pipeline is included. However, this input data includes the **Diabetic** label that the model predicts, which is not included in new patient data for which a diabetes prediction hasn't yet been made. Delete this module and replace it with an **Enter Data Manually** module, containing the following CSV data, which includes feature values without labels for three new patient observations, Double click on Enter Data Manually component and copy paste the following text in csv part:

```
PatientID,Pregnancies,PlasmaGlucose,DiastolicBloodPressure,TricepsThickness,  
SerumInsulin,BMI,DiabetesPedigree,Age  
1882185,9,104,51,7,24,27.36983156,1.350472047,43  
1662484,6,73,61,35,24,18.74367404,1.074147566,75  
1228510,4,115,50,29,243,34.69215364,0.741159926,59
```

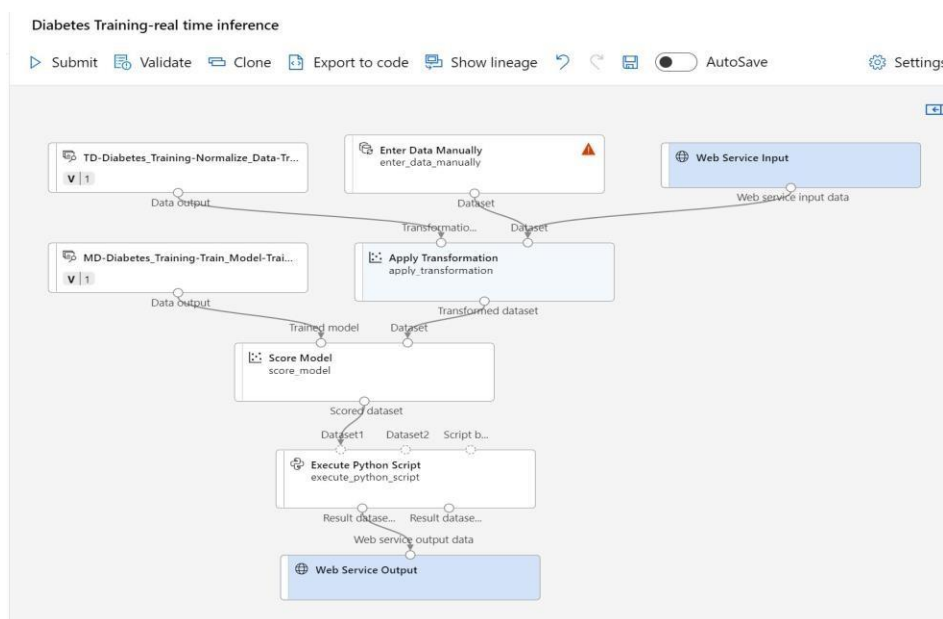
7. Connect the new **Enter Data Manually** module to the same **Dataset** input of the **Apply Transformation** module as the **Web Service Input**.
8. The inference pipeline includes the **Evaluate Model** module, which isn't useful when predicting from new data, so delete this module.
9. The output from the **Score Model** module includes all of the input features and the predicted label and probability score. To limit the output to only the prediction and probability:
- Delete the connection between the **Score Model** module and the **Web Service Output**.
 - Add an **Execute Python Script** module, double click on it and replace all of the default python script with the following code (which selects only the **PatientID**, **Scored Labels** and **Scored Probabilities** columns and renames them appropriately):

```
import pandas as pd
```

```
def azureml_main(dataframe1 = None, dataframe2 = None):
```

```
    scored_results = dataframe1[['PatientID', 'Scored Labels', 'Scored Probabilities']]
    scored_results.rename(columns={'Scored Labels':'DiabetesPrediction',
                                   'ScoredProbabilities':'Probabilt y'}, inplace=True)
    return scored_results
```

1. Connect the output from the **Score Model** module to the **Dataset1** (left-most) input of the **Execute Python Script**, and connect the first output of the **Execute Python Script** module to the **Web Service Output**.
2. Verify that your pipeline looks similar to the following image:



3. Run the pipeline as a new experiment named **mslearn-diabetes-inference** on your compute cluster. To do this, click on “Configure and Submit”, and create a new experiment and call it “mslearn-diabetes-inference”. Click on Next. For Runtime setting, select computer cluster you had created before and click on Review+Submit and then click Submit. The experiment may take a while to run.
4. When the pipeline has completed, open it from pipelines, then right click the **Execute Python Script** module. Select the **Preview data** and select **Result dataset** to see the predicted labels and probabilities for the three patient observations in the input data.

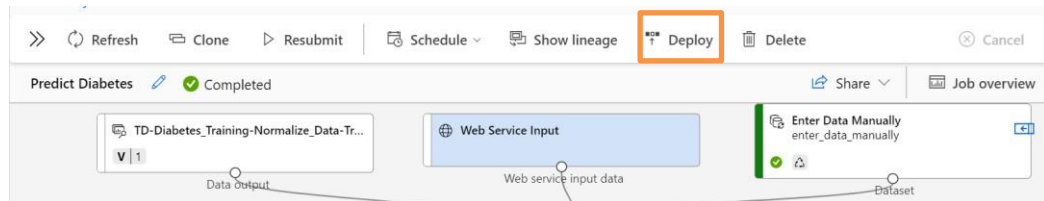
Your inference pipeline predicts whether or not patients are at risk for diabetes based on their features. Now you’re ready to publish the pipeline so that client applications can use it.

After you've created and tested an inference pipeline for real-time inferencing, you can publish it as a service for client applications to use.

Note In this exercise, you'll deploy the web service to an Azure Container Instance (ACI). This type of compute is created dynamically, and is useful for development and testing. For production, you should create an *inference cluster*, which provide an Azure Kubernetes Service (AKS) cluster that provides better scalability and security.

Deploy a service

1. View the **Predict Diabetes** inference pipeline you created in the previous part.
2. Click on it. In the new window, select **Deploy** and deploy a **new real-time endpoint**, using the following settings:
 - **Name:** predict-diabetes
 - **Description:** Classify diabetes
 - **Compute type:** Azure Container Instance



3. Wait for the web service to be deployed - **this can take several minutes**. The deployment status is shown at the top left of the designer interface.

Test the service

1. On the **Endpoints** page from left hand panel (under Assets), open the **predict-diabetes** real-time endpoint.
2. When the **predict-diabetes** endpoint opens, select the **Test** tab. We will use it to test our model with new data. Delete the current data under **Input data to test real-time endpoint**.

The screenshot shows the Azure Machine Learning studio interface. On the left is a sidebar with a navigation menu. Under the 'Assets' section, the 'Endpoints' option is highlighted with a yellow box. The main area displays the 'predict-diabetes' endpoint. At the top, there are tabs for 'Details', 'Test' (which is selected and highlighted with a yellow box), 'Consume', and 'Deployment logs'. Below the tabs, the title 'Input data to test real-time endpoint' is shown next to a blue 'Test' button. The main content area contains a JSON code editor with the following structure:

```
{
  "Inputs": {
    "WebServiceInput0": [
      {
        "PatientID": 1882185,
        "Pregnancies": 9,
        "PlasmaGlucose": 104,
        "DiastolicBloodPressure": 51,
        "TricepsThickness": 7,
        "SerumInsulin": 24,
        "BMI": 27.36983156,
        "DiabetesPedigree": 1.3504720469999998,
        "Age": 43
      }
    ]
  },
  "GlobalParameters": {}
}
```

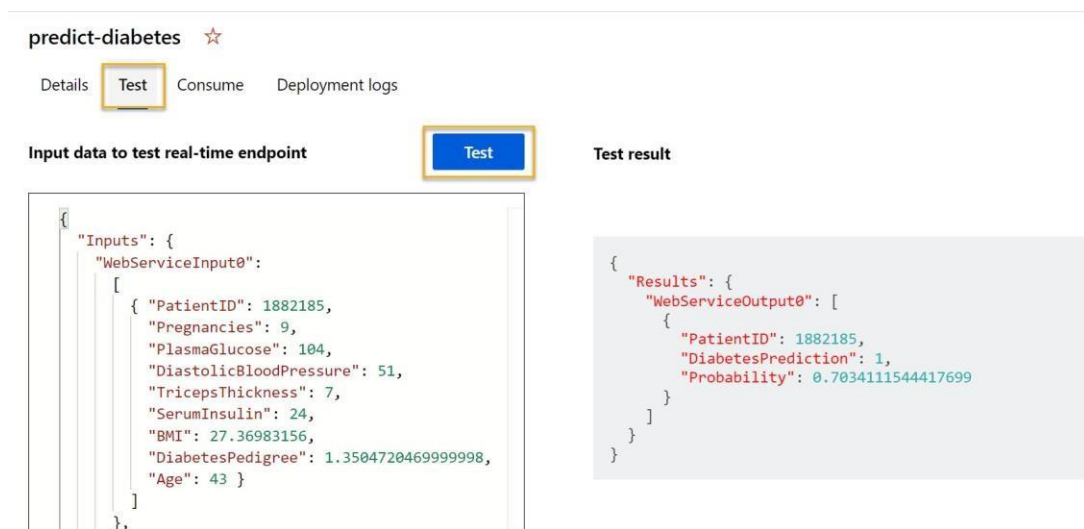
3. Copy and paste the below data into the data section: (When you are writing the code you should pay attention to indentation)

```
{
  "Inputs": {
    "input1": [
      {
        "PatientID": 1882185,
        "Pregnancies": 9,
        "PlasmaGlucose": 104,
        "DiastolicBloodPressure": 51,
        "TricepsThickness": 7,
        "SerumInsulin": 24,
        "BMI": 27.36983156,
        "DiabetesPedigree": 1.3504720469999998,
        "Age": 43
      }
    ]
  },
}
```

```
"GlobalParameters": {}  
}
```

Note The JSON above defines features for a patient, and uses the **predict-diabetes** service you created to predict a diabetes diagnosis.

1. Select **Test**. On the right hand of the screen, you should see the output **‘DiabetesPrediction’**. The output is 1 if the patient is predicted to have diabetes, and 0 if the patient is predicted not to have diabetes.



The screenshot shows the AWS API Gateway console for the **predict-diabetes** service. The **Test** tab is selected and highlighted with a yellow box. Below the tabs, there is a **Test** button, also highlighted with a yellow box. The **Input data to test real-time endpoint** section contains a JSON payload. The **Test result** section shows the output JSON.

Input data to test real-time endpoint:

```
{  
  "Inputs": {  
    "WebServiceInput0": [  
      {  
        "PatientID": 1882185,  
        "Pregnancies": 9,  
        "PlasmaGlucose": 104,  
        "DiastolicBloodPressure": 51,  
        "TricepsThickness": 7,  
        "SerumInsulin": 24,  
        "BMI": 27.36983156,  
        "DiabetesPedigree": 1.3504720469999998,  
        "Age": 43 }  
    ]  
  }  
}
```

Test result:

```
{  
  "Results": {  
    "WebServiceOutput0": [  
      {  
        "PatientID": 1882185,  
        "DiabetesPrediction": 1,  
        "Probability": 0.7034111544417699  
      }  
    ]  
  }  
}
```

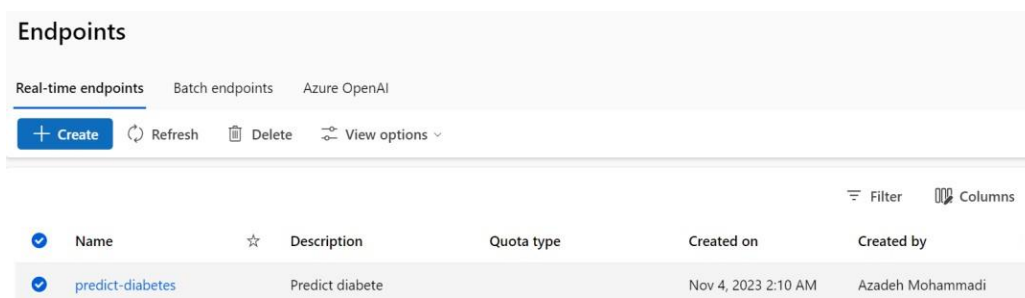
You have just tested a service that is ready to be connected to a client application using the credentials in the **Consume** tab.

Clean-up

Important:

The web service you created is hosted in an *Azure Container Instance*. If you don't intend to experiment with it further, you should delete the endpoint to avoid accruing unnecessary Azure usage.

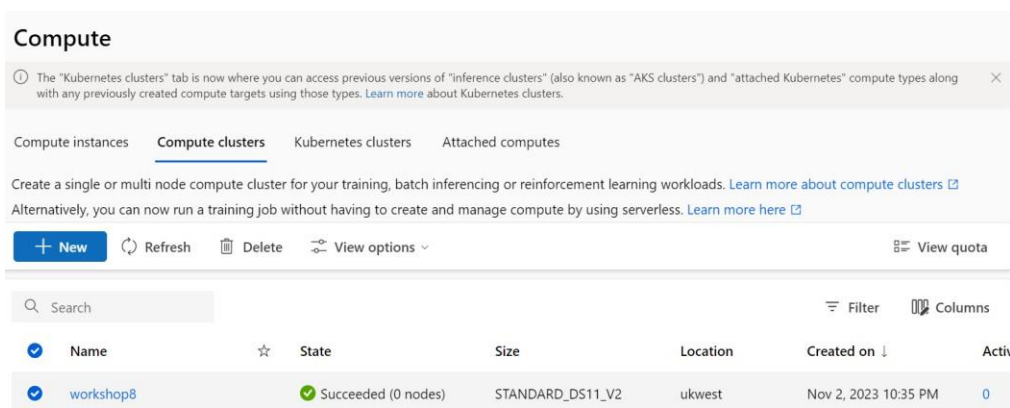
1. In [Azure Machine Learning studio](#), on the **Endpoints** tab, select the **predict-diabetes** endpoint. Then select **Delete** and confirm that you want to delete the endpoint.



The screenshot shows the 'Endpoints' section in the Azure Machine Learning studio. It has tabs for 'Real-time endpoints', 'Batch endpoints', and 'Azure OpenAI'. The 'Real-time endpoints' tab is active. Below the tabs are buttons for '+ Create', 'Refresh', 'Delete', and 'View options'. A table lists the endpoints with columns: Name, Description, Quota type, Created on, Created by, and a user icon. One endpoint is listed: 'predict-diabetes' with description 'Predict diabetes', created on 'Nov 4, 2023 2:10 AM', and created by 'Azadeh Mohammadi'.

Name	Description	Quota type	Created on	Created by	User
predict-diabetes	Predict diabetes		Nov 4, 2023 2:10 AM	Azadeh Mohammadi	N

2. On the **Compute** page, on the **Compute clusters** tab, select your compute cluster and then select **Delete**.



The screenshot shows the 'Compute' section in the Azure Machine Learning studio. It has tabs for 'Compute instances', 'Compute clusters', 'Kubernetes clusters', and 'Attached computes'. The 'Compute clusters' tab is active. Below the tabs are buttons for '+ New', 'Refresh', 'Delete', and 'View options'. A table lists the compute clusters with columns: Name, State, Size, Location, Created on, and Active. One cluster is listed: 'workshop8' with state 'Succeeded (0 nodes)', size 'STANDARD_DS11_V2', location 'ukwest', created on 'Nov 2, 2023 10:35 PM', and active count '0'.

Name	State	Size	Location	Created on	Active
workshop8	Succeeded (0 nodes)	STANDARD_DS11_V2	ukwest	Nov 2, 2023 10:35 PM	0

Note Stopping your compute ensures your subscription won't be charged for compute resources. You will however be charged a small amount for data storage as long as the Azure Machine Learning workspace exists in your subscription. If you have finished exploring Azure Machine Learning, you can delete the Azure Machine Learning workspace and associated resources. However, if you plan to complete any other labs in this series, you will need to recreate it.

To delete your workspace:

1. In the [Azure portal](#), in the **Resource groups** page, open the resource group you specified when creating your Azure Machine Learning workspace.
2. Click **Delete resource group**, type the resource group name to confirm you want to delete it, and select **Delete**.